OS Core Simulation - Complete Documentation

Project Overview

os-core-simulation is a lightweight operating system kernel simulator written in C. It simulates core OS components including process management, memory allocation, file system operations, interrupt handling, and CPU scheduling. This project is ideal for educational purposes to understand how operating systems manage resources at a fundamental level.

Repository: FatehAli02/os-core-simulation
Language: C (98.9%) + Makefile (1.1%)
License: Not specified

Table of Contents

Project Structure

```
os-core-simulation/
├── include/                      # Header files
│   ├── process.h                # Process management interface
│   ├── process_scheduler.h      # Scheduling algorithms
│   ├── memory.h                 # Memory manager interface
│   ├── file_system.h            # Filesystem interface
│   ├── syscall.h                # System call definitions
│   └── interrupt.h              # Interrupt handling interface
│
├── src/                         # Implementation files
│   ├── main.c                   # Entry point
│   ├── process/
│   │   ├── process.c            # Process creation and management
│   │   └── process_scheduler.c  # Scheduling implementation
│   ├── memory/
│   │   └── memory_manager.c     # Memory allocation/deallocation
│   ├── filesystem/
│   │   └── file_system.c        # File operations
│   ├── syscall/
```

```
│   │   └── syscall.c           # System call implementation
│   └── interrupt/
│       └── interrupt.c         # Interrupt queue and handling
│
├── tests/                      # Test files
│   └── main.c                  # Test entry point
│
├── Makefile                    # Build configuration
├── README.md                   # Basic project info
└── os_simulation.exe           # Compiled binary (Windows)
```

Core Components

1. Process Management (`process.c`, `process.h`)

Manages the complete lifecycle of processes and maintains process control blocks (PCBs).

Key Data Structures:

```
enum ProcessState {
    NEW,        // Process just created
    READY,      // Ready to run, waiting in queue
    RUNNING,    // Currently executing
    WAITING,    // Blocked on I/O
    TERMINATED  // Finished execution
};

struct PCB {
    int pid;              // Process identifier
    int burstTime;        // CPU time required
    int memoryUsage;      // Memory allocated
    enum ProcessState status;
    int priority;         // Process priority
};

struct Node {
    struct PCB pcb;
    struct Node* next;    // Linked list node
};
```

Core Functions:

Function	Purpose
`int processCreation(int priority, int burstTime, int memoryUsage)`	Creates new process with given parameters
`void terminateProcess(int pid)`	Removes process and frees associated resources

Function	Purpose
`int processExists(int pid)`	Validates if a process with given PID exists
`void processAdmit(int pid)`	Transitions process from NEW → READY state
`void processDispatch(int pid)`	Transitions process from READY → RUNNING state
`void processPreempt(int pid)`	Transitions process from RUNNING → READY (timer interrupt)
`void processBlock(int pid)`	Transitions process from RUNNING → WAITING (I/O request)
`void processWakeup(int pid)`	Transitions process from WAITING → READY (I/O complete)

Key Features:

- Linked-list based process management
- Safe PID generation with overflow handling
- Prevents duplicate PIDs using INT_MAX validation
- Complete state machine implementation

---

2. Memory Management (`memory_manager.c`, `memory.h`)

Implements dynamic memory allocation with multiple strategies and fragmentation tracking.

Configuration Constants:

```
#define TOTAL_MEMORY_SIZE 1024  // Total available memory: 1KB
#define MAX_BLOCKS 100          // Maximum memory blocks
```

Allocation Strategies:

- **STRATEGY_FIRST_FIT (1)**: Allocates first available block large enough for request
- **STRATEGY_BEST_FIT (2)**: Finds smallest block that fits (minimizes wasted space)
- **STRATEGY_WORST_FIT (3)**: Uses largest available block (maximizes remaining contiguous space)

Memory Block Structure:

```
struct MemoryBlock {
    int startAddress;  // Starting address in memory
    int size;          // Block size in bytes
    int pid;           // Owner process ID (0 if free)
    int isFree;        // 1 if free, 0 if allocated
};
```

Core Functions:

Function	Purpose
`void initializeMemory(void)`	Sets up single 1KB free block
`int allocateMemory(int pid, int size, int strategy)`	Allocates memory using specified strategy
`void deallocateMemory(int pid)`	Frees all memory blocks owned by process
`void printMemoryMap(void)`	Displays current memory layout and usage
`void printFragmentationStats(void)`	Shows fragmentation analysis

Key Features:

- Dynamic block splitting on partial allocations
- Automatic block merging on deallocation (coalescing)
- Fragmentation statistics tracking
- Comprehensive error handling for out-of-memory conditions

---

3. Process Scheduler (`process_scheduler.c`, `process_scheduler.h`)

Implements Multi-Level Feedback Queue (MLFQ) scheduling algorithm to balance interactive and batch workloads.

Configuration Constants:

```
#define _MLFQ_L 4           // 4 priority levels
#define _Q_BS 2             // Base time quantum (2 time units)
#define _BST_T 200          // Boost time threshold (200 time units)
#define _P_MAX 1024         // Maximum number of processes
```

Queue Architecture:

```
Level 0: Time Quantum = 2   (Highest priority, interactive processes)
Level 1: Time Quantum = 4
Level 2: Time Quantum = 8
Level 3: Time Quantum = 16  (Lowest priority, batch processes)
```

Core Functions:

Function	Purpose

Function	Purpose
`void sched_init(void)`	Initializes scheduler queues
`struct Node* sched_next(void)`	Returns next process to execute
`void sched_update(struct Node* p, int t)`	Updates process after time quantum expires
`void sched_boost(void)`	Boosts all processes to highest priority (starvation prevention)

Scheduling Algorithm:

1. Processes start at their assigned priority level
2. If a process uses its entire time quantum, it moves to lower priority
3. If a process yields early (I/O), it remains at current level
4. Periodic boost resets all processes to highest priority (prevents starvation)
5. Round-robin scheduling within each priority level

---

4. File System (`file_system.c`, `file_system.h`)

Implements a simple in-memory file system with permission-based access control.

File Structure:

```
typedef struct FileNode {
    char name[32];           // Filename (max 31 characters)
    int permissions;         // Bitmask: FS_READ | FS_WRITE
    char *content;           // Dynamic content buffer
    size_t size;             // Content size in bytes
    struct FileNode *next;   // Linked list pointer
} FileNode;
```

Permission Flags:

```
#define FS_READ  0x1   // Read permission (001 in binary)
#define FS_WRITE 0x2   // Write permission (010 in binary)
// Combined: FS_READ | FS_WRITE = 0x3 (Read and Write)
```

Core Functions:

Function	Purpose
`void fs_init(void)`	Initializes filesystem
`int fs_createFile(const char *name, int permissions)`	Creates new file with permissions

Function	Purpose
`int fs_writeFile(const char *name, const char *data)`	Writes data to file
`int fs_readFile(const char *name)`	Reads and displays file content
`int fs_deleteFile(const char *name)`	Deletes file and frees memory
`void fs_listFiles(void)`	Lists all files in system
`int getFileCount(void)`	Returns number of files

Key Features:

- Filename validation (1-31 characters)
- Duplicate filename prevention
- Permission checking before read/write operations
- Dynamic content storage with automatic cleanup
- Memory allocation failure detection

---

5. Interrupt System (`interrupt.c`, `interrupt.h`)

Ring buffer-based interrupt queue for hardware events and I/O completion.

Interrupt Event Structure:

```
typedef enum {
    INTERRUPT_TIMER,   // Timer/scheduler interrupt
    INTERRUPT_IO,      // I/O completion interrupt
    INTERRUPT_SYSCALL  // System call interrupt
} InterruptType;

typedef struct {
    InterruptType type;
    int pid;           // Target process ID
    int data;          // Additional interrupt data
    time_t timeStamp;  // When interrupt occurred
} InterruptEvent;
```

Queue Configuration:

```
#define INTERRUPT_QUEUE_SIZE 64  // Ring buffer capacity
```

Core Functions:

Function	Purpose
`int interruptInit(void)`	Initializes interrupt queue

Function	Purpose
`int interruptRaise(InterruptType type, int pid, int data)`	Enqueues interrupt event
`int interruptFetch(InterruptEvent *event)`	Dequeues and returns interrupt
`void interruptHandle(const InterruptEvent *event)`	Processes interrupt event
`int interruptIsEmpty(void)`	Checks if queue is empty
`int interruptIsFull(void)`	Checks if queue is full

Key Features:

- FIFO ring buffer implementation
- Timestamp recording for each interrupt
- Head/tail pointer management
- Full/empty status checking
- Overflow protection

---

6. System Calls (`syscall.c`, `syscall.h`)

Unified interface for all OS operations. Provides input validation and coordinates between subsystems.

System Call Categories:

Process Operations

```
int sys_create_process(int priority, int burstTime, int memSize)
// Creates new process with memory allocation
// Returns: PID on success, -1 on failure

int sys_terminate_process(int pid)
// Terminates process and frees all resources
// Returns: 0 on success, -1 on failure

int sys_get_process_list(void)
// Displays table of all processes with current status
// Returns: 0 on success, -1 on failure
```

Memory Operations

```
int sys_allocate_memory(int pid, int size, int strategy)
// Allocates additional memory to existing process
// Returns: Memory address on success, -1 on failure

int sys_deallocate_memory(int pid)
// Frees all memory allocated to process
// Returns: 0 on success, -1 on failure
```

```
int sys_get_memory_map(void)
// Displays current memory layout and fragmentation stats
// Returns: 0 on success, -1 on failure
```

File System Operations

```
int sys_create_file(const char* name, int permissions)
int sys_write_file(const char* name, const char* data)
int sys_read_file(const char* name)
int sys_delete_file(const char* name)
void sys_list_files(void)
```

Initialization & Interrupt Handling

```
void sys_init(void)
// Initializes all subsystems:
// - scheduler
// - memory manager
// - filesystem
// - interrupt queue

void sys_execute_interrupts(void)
// Processes all pending interrupts in queue

int sys_scheduler_timer(int pid)
// Hardware timer interrupt handler
```

Building & Running

Prerequisites

- **GCC Compiler**: For C code compilation
- **Make**: For build automation
- **Windows/Linux/macOS**: Cross-platform support

Build Instructions

```
# Clone the repository
git clone https://github.com/FatehAli02/os-core-simulation.git
cd os-core-simulation

# Build the project
make
```

```
# Clean build artifacts
make clean
```

Makefile Configuration

The Makefile automatically detects the operating system:

Windows:

```
RM = del
TARGET = os_simulation.exe
```

Linux/macOS:

```
RM = rm -f
TARGET = os_simulation
```

Compilation Variables:

```
CC = gcc
CFLAGS = -Wall -Wextra -I./include

SOURCES = src/filesystem/file_system.c \
          src/interrupt/interrupt.c \
          src/memory/memory_manager.c \
          src/process/process_scheduler.c \
          src/process/process.c \
          src/syscall/syscall.c \
          tests/main.c \
          src/main.c
```

Running the Simulation

```
# Windows
os_simulation.exe

# Linux/macOS
./os_simulation
```

System Architecture

Initialization Flow

```
main()
  ↓
sys_init()
  ├─ sched_init()           Initialize MLFQ scheduler
  ├─ initializeMemory()     Setup 1KB memory pool
  ├─ fs_init()              Initialize filesystem
  └─ interruptInit()        Setup interrupt queue
  ↓
Main Loop (Event Processing)
```

Operation Flow

```
User Request / System Call
  ↓
sys_[operation]() validation layer
  ├─ Input validation
  ├─ Permission checking
  └─ Call appropriate subsystem
  ↓
Subsystem Processing
  ├─ Perform operation
  └─ Return result

Hardware Event (Timer/I/O)
  ↓
interruptRaise()
  ├─ Enqueue in ring buffer
  └─ Raise pending flag

Main Loop
  ↓
sys_execute_interrupts()
  ├─ Fetch all pending interrupts
  └─ Process each with interruptHandle()
```

Process State Transitions

```
│       ├─ processBlock()
│       │   ↓
│       │  WAITING
│       │   (I/O)
│       │   ↓
│       └─ processWakeup()
│           ↓
└──────→ Returns to READY

        RUNNING
          ↓
 processTerminate()
          ↓
      TERMINATED
          ↓
        EXIT
```

Key Features

✅ Process Management

- Process creation with configurable priority and burst time
- Safe PID generation and management
- Complete process state tracking (NEW → READY → RUNNING → WAITING → TERMINATED)
- Linked-list based process queue

✅ CPU Scheduling

- Multi-level feedback queue (MLFQ) algorithm
- Adaptive priority system based on process behavior
- Starvation prevention through periodic priority boosting
- Round-robin scheduling within priority levels
- Time quantum allocation based on priority

✅ Memory Management

- Three allocation strategies (First-Fit, Best-Fit, Worst-Fit)
- Dynamic memory block splitting on partial allocations
- Automatic memory coalescing (block merging)
- Fragmentation analysis and reporting
- Automatic memory deallocation on process termination

✅ File System

- Simple in-memory filesystem with linked-list storage
- Permission-based access control (read, write)
- Dynamic file content storage
- File enumeration and deletion
- Filename validation and duplicate prevention

✅ Interrupt Handling

- Ring buffer-based interrupt queue (64 entries)
- Multiple interrupt types (Timer, I/O, Syscall)
- Timestamp recording for each interrupt
- FIFO processing with overflow protection
- Event-driven architecture

✅ System Calls

- Unified interface for all OS operations
- Comprehensive input validation
- Error handling and reporting
- Cross-module coordination
- Resource tracking and management

API Reference

Process Module (`process.h`)

```
struct Node* getProcessHead(void);
const char* getStateString(enum ProcessState s);
int processCreation(int priority, int burstTime, int memoryUsage);
void terminateProcess(int pid);
int processExists(int pid);
void processAdmit(int pid);
void processDispatch(int pid);
void processPreempt(int pid);
void processBlock(int pid);
void processWakeup(int pid);
```

Memory Module (`memory.h`)

```
void initializeMemory(void);
int allocateMemory(int pid, int size, int strategy);
void deallocateMemory(int pid);
void printMemoryMap(void);
void printFragmentationStats(void);
```

Scheduler Module (`process_scheduler.h`)

```
void sched_init(void);
struct Node* sched_next(void);
void sched_update(struct Node* p, int t);
void sched_boost(void);
```

File System Module (`file_system.h`)

```
void fs_init(void);
int fs_createFile(const char *name, int permissions);
int fs_writeFile(const char *name, const char *data);
int fs_readFile(const char *name);
int fs_deleteFile(const char *name);
void fs_listFiles(void);
int getFileCount(void);
```

Interrupt Module (`interrupt.h`)

```
int interruptInit(void);
int interruptRaise(InterruptType type, int pid, int data);
int interruptFetch(InterruptEvent *event);
void interruptHandle(const InterruptEvent *event);
int interruptIsEmpty(void);
int interruptIsFull(void);
```

System Call Module (`syscall.h`)

```
void sys_init(void);
int sys_create_process(int priority, int burstTime, int memSize);
int sys_terminate_process(int pid);
int sys_allocate_memory(int pid, int size, int strategy);
int sys_deallocate_memory(int pid);
int sys_get_process_list(void);
int sys_get_memory_map(void);
int sys_create_file(const char* name, int permissions);
int sys_write_file(const char* name, const char* data);
int sys_read_file(const char* name);
int sys_delete_file(const char* name);
void sys_list_files(void);
void sys_execute_interrupts(void);
int sys_scheduler_timer(int pid);
```

Usage Examples

Example 1: Creating and Managing Processes

```
#include "syscall.h"

int main() {
```

```
    sys_init();  // Initialize all subsystems

    // Create process: priority=1, burst_time=100, memory=256 bytes
    int pid = sys_create_process(1, 100, 256);

    if (pid != -1) {
        printf("Process created with PID: %d\n", pid);

        // Display all processes
        sys_get_process_list();

        // Terminate when done
        sys_terminate_process(pid);
    }

    return 0;
}
```

Example 2: Memory Allocation with Different Strategies

```
// Create process with initial memory
int pid = sys_create_process(2, 100, 256);

// Allocate additional memory using First-Fit strategy
int addr1 = sys_allocate_memory(pid, 128, STRATEGY_FIRST_FIT);
printf("Allocated 128 bytes at address: %d\n", addr1);

// Allocate using Best-Fit (minimizes waste)
int addr2 = sys_allocate_memory(pid, 64, STRATEGY_BEST_FIT);
printf("Allocated 64 bytes at address: %d\n", addr2);

// Display memory layout
sys_get_memory_map();

// Cleanup
sys_deallocate_memory(pid);
sys_terminate_process(pid);
```

Example 3: File System Operations

```
#include "syscall.h"

void file_demo() {
    sys_init();

    // Create files with different permissions
    sys_create_file("config.txt", FS_READ | FS_WRITE);  // Read and Write
    sys_create_file("readme.txt", FS_READ);              // Read only
```

```
    // Write data
    sys_write_file("config.txt", "timeout=30\nretries=3");
    sys_write_file("readme.txt", "Usage: ./os_simulation");

    // Read files
    printf("=== Reading config.txt ===\n");
    sys_read_file("config.txt");

    printf("\n=== Reading readme.txt ===\n");
    sys_read_file("readme.txt");

    // List all files
    printf("\n=== File Listing ===\n");
    sys_list_files();

    // Cleanup
    sys_delete_file("config.txt");
    sys_delete_file("readme.txt");
}
```

Example 4: Interrupt Handling

```
#include "interrupt.h"
#include "syscall.h"

void interrupt_demo() {
    sys_init();

    // Create a process
    int pid = sys_create_process(1, 50, 256);

    // Raise interrupt events
    interruptRaise(INTERRUPT_TIMER, pid, 0);
    interruptRaise(INTERRUPT_IO, pid, 256);

    // Process all pending interrupts
    sys_execute_interrupts();

    // Cleanup
    sys_terminate_process(pid);
}
```

Technical Details

Memory Layout Example

Initial State:

```

```

```
┌──────────────────────────────────────────┐
│  Free Block (1024 bytes)                 │
│  Address: 0-1023                         │
└──────────────────────────────────────────┘
```

After Allocating 256 bytes to PID 1:

```
┌──────────────────────┬──────────────────────────────────┐
│ PID 1: 256 bytes     │ Free: 768 bytes                  │
│ Address: 0-255       │ Address: 256-1023                │
└──────────────────────┴──────────────────────────────────┘
```

After Allocating 128 bytes to PID 2:

```
┌──────────────────────┬──────────────────┬────────────────────────┐
│ PID 1: 256 bytes     │ PID 2: 128 b     │ Free: 640 bytes        │
│ Address: 0-255       │ Addr: 256-383    │ Address: 384-1023      │
└──────────────────────┴──────────────────┴────────────────────────┘
```

Process Scheduling Example

MLFQ with 4 Priority Levels:

```
Initial Setup:
Level 0: TQ = 2   (High priority interactive)
Level 1: TQ = 4
Level 2: TQ = 8
Level 3: TQ = 16  (Low priority batch)

Process Flow Example:
- Process A starts at Level 0, TQ = 2
- Uses full TQ without blocking → moves to Level 1
- Uses full TQ at Level 1 → moves to Level 2
- Continues until Level 3
- Every 200 time units → BOOST (all back to Level 0)
```

PID Generation Algorithm

```
Strategy:
1. First attempt: Use processIdCount (0, 1, 2, ...)
2. If INT_MAX reached: Scan for gaps in used PIDs
3. Fallback: Linear search from 1 until INT_MAX

Benefits:
```

```
✓ O(1) allocation in typical case
✓ Prevents PID exhaustion for long-running systems
✓ No special cleanup on integer overflow
```

File System Permissions Model

```
// Permission encoding using bitwise flags
FS_READ = 0x1   // Binary: 001
FS_WRITE = 0x2  // Binary: 010

// Example usage:
FS_READ           // 0x1 (001) = Read-only
FS_WRITE          // 0x2 (010) = Write-only
FS_READ | FS_WRITE // 0x3 (011) = Read and Write

// Permission checking:
if (permissions & FS_READ) {
    // Allow read operation
}
if (permissions & FS_WRITE) {
    // Allow write operation
}
```

Interrupt Ring Buffer

Ring Buffer Structure:

```
Configuration:
- Size: 64 entries maximum
- Structure: FIFO queue
- Implementation: Head/tail pointers + count

Example Timeline:

Initial: head=0, tail=0, count=0 (empty)
┌─────┬─────┬─────┬─────┬─────┐
│     │     │     │     │ ... │
└─────┴─────┴─────┴─────┴─────┘

After 3 interrupts: head=0, tail=3, count=3
┌─────┬─────┬─────┬─────┬─────┐
│INT1│INT2│INT3│     │ ... │
└─────┴─────┴─────┴─────┴─────┘

After processing INT1: head=1, tail=3, count=2
┌─────┬─────┬─────┬─────┬─────┐
│     │INT2│INT3│     │ ... │
└─────┴─────┴─────┴─────┴─────┘
```

Limitations & Observations

⚠ Current Issues

1. **Incomplete Implementation**:
    - `src/main.c` is empty
    - `tests/` directory is empty
    - No actual simulation loop implemented
2. **Hardcoded Limits**:
    - Memory: 1KB (fixed)
    - Interrupt queue: 64 entries
    - Memory blocks: 100 max
    - Processes: 1024 max
    - Filename length: 31 characters
3. **In-Memory Only**:
    - File system is entirely in-memory
    - No persistence to disk
    - Data lost on program termination
4. **Single-User Model**:
    - No multi-user support
    - No privilege levels
    - No user-specific permissions
5. **No Virtual Memory**:
    - No swapping or paging
    - Limited to physical 1KB
    - No page tables or virtual address spaces
6. **Scheduler Partially Implemented**:
    - MLFQ macros defined but logic may need verification

Recommendations for Enhancement

High Priority

- [ ] Implement complete `main.c` with interactive menu/CLI
- [ ] Add comprehensive unit tests in `tests/` directory
- [ ] Implement disk-based file persistence
- [ ] Add configuration file support for system parameters

Medium Priority

- [ ] Implement virtual memory with paging
- [ ] Add multi-user authentication model
- [ ] Create visualization tools (ASCII or GUI dashboard)
- [ ] Performance profiling and optimization
- [ ] Add detailed logging and debugging output

Low Priority

- [ ] Web-based interface for remote management
- [ ] Distributed simulation across multiple machines
- [ ] Machine learning-based scheduler optimization
- [ ] Integration with existing OS kernels
- [ ] Support for real hardware device simulation

Conclusion

The **os-core-simulation** project provides a hands-on educational platform for understanding fundamental operating system concepts:

- **Process Management**: How OSes create, schedule, and manage processes
- **Memory Management**: Allocation strategies and fragmentation handling
- **CPU Scheduling**: Priority-based scheduling algorithms (MLFQ)
- **File Operations**: Persistent storage abstraction
- **Interrupt Handling**: Event-driven architecture
- **System Calls**: Unified OS interface

The modular architecture makes it ideal for:

- 📚 Educational purposes (OS courses)
- 🔬 Research and experimentation
- 🛠️ Understanding kernel internals
- 🎓 Student projects and assignments

For questions or contributions, refer to the repository's Issues and Pull Requests sections.

Last Updated: April 10, 2026
Author: FatehAli02
Repository: https://github.com/FatehAli02/os-core-simulation